Standalone Bridge EA Design

Bridge_MT4_File.mq4 – File-Driven Execution for MT4

---

1. Purpose

`Bridge_MT4_File.mq4` is a **standalone Expert Advisor** for MetaTrader 4 that:

- Periodically scans a folder on disk for **action files** created by an external application.
- Parses each action, validates it against configurable risk constraints, and executes it using MT4 trade functions.
- Writes a **feedback file** for each processed action so the external app can track fills/errors.
- Can be used **independently** (you can test it with manually created files) or together with your Python trading engine.

It turns MT4 into a **generic file-driven order executor**.

---

2. High-Level Behaviour

1. On initialization:
    - Set up input parameters.
    - Ensure folders exist (or at least log paths).
    - Set a timer (e.g. 1 second) to poll regularly.
2. On timer:
    - List files in the outgoing folder (e.g. `C:\bridge\outgoing\`).
    - For each file:
        - Parse into a key/value map.
        - Validate asset, size, spread, time validity.
        - Optionally ask for confirmation.
        - Execute the specified action (OPEN, CLOSE_ALL, etc.).
        - Write a feedback file into the incoming folder.
        - Move the processed file to an archive folder or delete it.
3. On deinitialization:
    - Kill the timer.
    - Close any file handles.

This EA does **not** need to know anything about Python; it just follows a file protocol.

---

3. Inputs and Configuration

Define these inputs in `Bridge_MT4_File.mq4`:

- Paths:
  - `extern string BridgeFolder = "C:\\bridge\\outgoing\\";`
  - `extern string FeedbackFolder = "C:\\bridge\\incoming\\";`
  - `extern string ArchiveFolder = "C:\\bridge\\archive\\";`
- Execution control:
  - `extern bool OnlyCurrentSymbol = false;`
  - `extern bool AskForConfirmation = false;`
  - `extern double MaxLotsPerTrade = 1.0;`
  - `extern double MaxSpread = 3.0;` // in points or pips depending on your broker
  - `extern int MagicNumberBase = 202600;`
  - `extern int PollIntervalSeconds = 1;`
- Logging:
  - `extern bool VerboseLogging = true;`

You can always extend these later.

---

4. Action File Format

Use a simple **key=value** text format so you can easily create and parse files.

Example file (saved as `C:\bridge\outgoing\xauusd_20260523_120001_001.txt`):

```
text
```

```
id=xauusd_20260523_120001_001
asset=XAUUSD
action=OPEN
side=BUY
size=0.10
order_type=MARKET
sl=2380.50
```

```
tp=2405.00
comment=OB_TREND_v1
magic_number=202605
valid_until=2026-05-23T12:05:00Z
```

Another example for closing all orders on a symbol:

```
text
```

```
id=xauusd_20260523_120010_002
asset=XAUUSD
action=CLOSE_ALL
side=
size=
order_type=
sl=
tp=
comment=risk_engine_daily_limit
magic_number=202605
valid_until=2026-05-23T12:15:00Z
```

Required keys:

- `id` – unique identifier for this action.
- `asset` – symbol, e.g. `XAUUSD`.
- `action` – at minimum:
  - `OPEN`
  - `CLOSE_ALL`
- `side` – `BUY` or `SELL` (for OPEN).
- `size` – lots (for OPEN).
- `order_type` – `MARKET` (you can extend later).
- `sl` – stop loss price or blank.
- `tp` – take profit price or blank.
- `magic_number` – integer; if blank, EA uses `MagicNumberBase`.
- `valid_until` – ISO string or blank (if blank, always valid).

You can ignore unknown keys.

---

5. Feedback File Format

After processing an action, the EA writes a feedback file in `FeedbackFolder`, e.g. `C:\bridge\incoming\xauusd_20260523_120001_001_result.txt`:

For successful OPEN:

```
text
```

```
id=xauusd_20260523_120001_001
status=FILLED
asset=XAUUSD
action=OPEN
side=BUY
size=0.10
tickets=12345678
avg_price=2392.40
message=OrderSend OK
error_code=0
```

For rejected action:

```
text
```

```
id=xauusd_20260523_120001_001
status=REJECTED
asset=XAUUSD
action=OPEN
side=BUY
size=0.10
tickets=
avg_price=
message=MaxSpreadExceeded
error_code=1
```

You can define simple error codes (e.g. 1 = local risk, 2 = MT4 error, etc.).

---

6. EA Internals

6.1 Initialization (`OnInit`)

- Validate folder strings:
  - Ensure they end with `\\`.
- Optionally check if folders exist using `FileIsExist` on a known test file or just log the path.
- Set timer:

```
text

int OnInit()
{
   // Logging
   Print("Bridge_MT4_File initialized. Outgoing: ",
BridgeFolder,
         ", Incoming: ", FeedbackFolder);

   // Set timer
   EventSetTimer(PollIntervalSeconds);
   return(INIT_SUCCEEDED);
}
```

6.2 Deinitialization (`OnDeinit`)

```
text

void OnDeinit(const int reason)
{
   EventKillTimer();
   Print("Bridge_MT4_File deinitialized.");
}
```

6.3 Timer handler (`OnTimer`)

Core steps:

1. Build a list of files in `BridgeFolder`.
   - In MQL4 you can use `FileFindFirst` / `FileFindNext` with `FILE_READ | FILE_TXT`.
2. For each file:

- Ignore `./..` or archive subfolder.
- Call a helper `ProcessActionFile(filename)`.

Pseudo-code:

```
text

void OnTimer()
{
   string mask = BridgeFolder + "*.txt";
   int handle = FileFindFirst(mask, filename, file_attr);
   if (handle != INVALID_HANDLE)
   {
      do
      {
         string fullpath = BridgeFolder + filename;
         ProcessActionFile(fullpath);
      }
      while (FileFindNext(handle, filename, file_attr));
      FileFindClose(handle);
   }
}
```

---

7. Parsing an Action File

7.1 Helper to read key=value pairs

Implement:

```
text

bool ReadActionFile(string path, string &id, string &asset,
string &action,
                    string &side, double &size, string
&ordertype,
                    double &sl, double &tp, int &magic, string
&valid_until,
```

```
                    string &comment)
{
   int handle = FileOpen(path, FILE_READ|FILE_TXT, ';');
   if(handle == INVALID_HANDLE)
   {
      Print("Failed to open action file: ", path);
      return(false);
   }

   string line;
   while(!FileIsEnding(handle))
   {
      line = FileReadString(handle);
      if(StringLen(line) == 0) continue;

      string parts[];
      int n = StringSplit(line, '=', parts);
      if(n < 2) continue;

      string key = StringTrim(parts[0]);
      string val = StringTrim(parts[1]);

      if(key == "id") id = val;
      else if(key == "asset") asset = val;
      else if(key == "action") action = val;
      else if(key == "side") side = val;
      else if(key == "size") size = StrToDouble(val);
      else if(key == "order_type") ordertype = val;
      else if(key == "sl") sl = (StringLen(val) > 0 ?
StrToDouble(val) : 0.0);
      else if(key == "tp") tp = (StringLen(val) > 0 ?
StrToDouble(val) : 0.0);
      else if(key == "magic_number") magic = (StringLen(val) > 0
? StrToInteger(val) : 0);
      else if(key == "valid_until") valid_until = val;
```

```
      else if(key == "comment") comment = val;
   }

   FileClose(handle);
   return(true);
}
```

You'll need to add `StringTrim` utility or manual trimming logic as MQL4 doesn't have it built-in.

7.2 Parsing and validation

In `ProcessActionFile`:

- Call `ReadActionFile`.
- Check required fields (at least `id`, `action`, `asset`).
- If `OnlyCurrentSymbol` is true, ensure `asset == Symbol()`.
- If `size > MaxLotsPerTrade` for OPEN, reject.
- Check spread:

```
text
```

```
double spread_points = (Ask - Bid) / Point;
if(spread_points > MaxSpread)
{
   // reject
}
```

- Validate `valid_until` if provided:
    - You can ignore or implement simple time comparison (optional).

If validation fails, write a **REJECTED** feedback and archive/delete the file.

---

8. Executing Actions

8.1 OPEN action

For `action == "OPEN"` and `side == "BUY"` or `SELL`:

- Check `ordertype == "MARKET"` (ignore others for now).
- If `AskForConfirmation` is true, show a confirmation via `MessageBox`:
  - If user clicks Cancel, do not trade; mark as REJECTED with message `UserCancelled`.
- Determine symbol:
  - `string symbol = OnlyCurrentSymbol ? Symbol() : asset;`
- Determine magic number:
  - If `magic > 0` from file, use it.
  - Else use `MagicNumberBase`.
- Call `RefreshRates()` before sending order.
- For BUY:
  - `OrderSend(symbol, OP_BUY, size, Ask, Slippage, sl, tp, comment, magic, 0, clrBlue);`
- For SELL:
  - `OrderSend(symbol, OP_SELL, size, Bid, Slippage, sl, tp, comment, magic, 0, clrRed);`
- Check return:
  - If `ticket > 0`, success:
    - Write `FILLED` feedback with ticket and price.
  - Else:
    - Capture `GetLastError()`:
      - Write `REJECTED` feedback with error code and message.

8.2 CLOSE_ALL action

For `action == "CLOSE_ALL"`:

- Iterate orders from `OrdersTotal()-1` down to `0`:
  - `OrderSelect(i, SELECT_BY_POS, MODE_TRADES).`
  - Filter:
    - `OrderSymbol() == asset` (or `OnlyCurrentSymbol ? Symbol() : asset`).
    - Optionally check `OrderMagicNumber()` if you want to only close your strategy's orders.
  - For `OP_BUY` → close at Bid.
  - For `OP_SELL` → close at Ask.
- Count:
  - number of orders closed successfully,
  - number failed.
- If everything closed or some closed, write `FILLED` with some summary.

- If nothing matched/closed, write `REJECTED` with message `NoOrdersToClose`.

You can later add actions like `CLOSE_WINNERS`, `CLOSE_LOSERS` etc. mirroring your existing scripts.

---

9. Writing Feedback and Archiving

9.1 Feedback file

Implement:

text

```
void WriteFeedback(string id, string asset, string action,
                   string status, string side, double size,
                   string tickets, double avg_price,
                   string message, int error_code)
{
   string filename = FeedbackFolder + id + "_result.txt";
   int handle = FileOpen(filename, FILE_WRITE|FILE_TXT);
   if(handle == INVALID_HANDLE)
   {
      Print("Failed to write feedback file: ", filename);
      return;
   }

   FileWrite(handle, "id=" + id);
   FileWrite(handle, "status=" + status);
   FileWrite(handle, "asset=" + asset);
   FileWrite(handle, "action=" + action);
   FileWrite(handle, "side=" + side);
   FileWrite(handle, "size=" + DoubleToString(size, 2));
   FileWrite(handle, "tickets=" + tickets);
   FileWrite(handle, "avg_price=" + (avg_price > 0 ?
DoubleToString(avg_price, Digits) : ""));
   FileWrite(handle, "message=" + message);
```

```
   FileWrite(handle, "error_code=" +
IntegerToString(error_code));

   FileClose(handle);
}
```

9.2 Archive or delete action file

After writing feedback:

- If you want to archive:

```
text
```

```
string dest = ArchiveFolder + filename_only;
bool moved = FileMove(path, dest);
if(!moved) FileDelete(path);
```

- Else just `FileDelete(path)`.

This prevents actions being processed twice.

---

10. Testing Without Python

You can test the EA completely standalone:

1. Set up folders:
   - `C:\bridge\outgoing\`
   - `C:\bridge\incoming\`
   - `C:\bridge\archive\`
2. Install `Bridge_MT4_File.mq4` into MT4:
   - Copy to `MQL4\Experts`.
   - Compile in MetaEditor.
   - Attach to a chart on a **demo** account.
3. In Windows Explorer:
   - Create a small `.txt` file in `C:\bridge\outgoing\` with:
     - `action=OPEN`, etc.
4. Watch:
   - EA log in MT4 (Experts tab) for messages.
   - Orders list for new trades.

- `C:\bridge\incoming\` for feedback files.
- `C:\bridge\archive\` for moved action files.

Once this works, you know the EA is good.

---

11. Connecting Later to Python (Optional)

Once `Bridge_MT4_File` is ready:

- In Python, implement a small helper to write the same `key=value` files:

```python
def write_action_file(path, **kwargs):
    lines = [f"{k}={v}" for k, v in kwargs.items()]
    tmp = path + ".tmp"
    with open(tmp, "w") as f:
        f.write("\n".join(lines))
    os.replace(tmp, path)
```

- The Python engine just calls this with the right keys, and the EA will handle execution exactly like your manual test files.

---

12. Summary

This document gives you **everything in one place** to:

- Design `Bridge_MT4_File.mq4` as a **standalone, reusable EA**.
- Define a simple, robust file protocol for actions and feedback.
- Test it manually, separate from your Python engine.
- Later plug your Python engine into the same protocol without changing the EA.

You can now build this EA as a separate project, keep this spec as its README/spec file, and once it's working, connect it to your broader multi-asset system.